Taller #3 · Resumen del código, celda por celda

Respuesta a una campaña bancaria · Aprendizaje Automático · Diplomado en Data Science UC 2026 · Notebook Grupo_XX_Taller3.ipynb (Python, Google Colab)

Idea general

El objetivo es predecir, *antes* de llamar, qué clientes tienen más probabilidad de contratar un depósito a plazo, elegir un buen modelo con validación cruzada, fijar un umbral según costos y luego describir perfiles (PCA, K-means) y registros inusuales (Mahalanobis). Tres principios atraviesan todo el código:

- **No hacer trampa con la información (sin fuga):** todo lo que "aprende" de los datos (medias para estandarizar, categorías, columnas seleccionadas) se ajusta solo con las filas de ajuste de cada pliegue, nunca con las de validación.
- **Comparar con predicciones fuera de pliegue (OOF):** cada registro se predice con un modelo que no lo vio. Así las métricas reflejan desempeño en datos nuevos.
- **El conjunto de prueba se usa una sola vez**, al final, después de haber elegido modelo y umbral solo con OOF.

Celda	Qué resuelve
Colab	Subir / encontrar los CSV
Preparación	Librerías, semilla, versiones, lectura
1(a)–1(c)	Definir el problema, explorar y preparar los datos
2(a)–2(c)	Selección de variables y 8 modelos con CV
3(a)–3(b)	Umbral por costo y evaluación final en prueba
4(a)–4(b)	PCA y K-means con 4 variables
5(a)–5(b)	Anomalías con distancia de Mahalanobis

0. Celdas iniciales

Celda de Google Colab (subir archivos)

Qué hace el código

- Detecta si el notebook corre en Colab (intenta `import google.colab`).
- Si faltan `clientes_entrenamiento.csv` o `clientes_prueba.csv`, abre el diálogo `files.upload()` y renombra los archivos si el navegador les agregó sufijos.
- Imprime qué archivos quedaron disponibles.

Por qué

- Colab parte cada sesión vacía; esta celda evita errores de "archivo no encontrado" y permite usar rutas relativas, como pide el enunciado.

Preparación (librerías, semilla y lectura)

Qué hace el código

- Importa numpy, pandas, scikit-learn, scipy y matplotlib, y todas las funciones que se usarán (OneHotEncoder, StandardScaler, SelectKBest, los modelos, PCA, KMeans y las métricas).
- Fija la semilla 2026 (`np.random.seed` y `SEMILLA`, que luego se pasa como `random_state` a cada modelo).
- Lee solo el entrenamiento e imprime las versiones de Python y de los paquetes.

Por qué

- La semilla hace los resultados reproducibles; informar versiones es un requisito de entrega; la prueba se deja cerrada hasta 3(b) para no contaminar las decisiones.

Pregunta 1 · Problema, preparación y representación

1(a) Definir tarea, unidad y respuesta

Qué hace el código

- Define la respuesta `contrata` y la lista de columnas excluidas: `duracion`, `id_cliente`, `pliegue` y la propia respuesta.
- Arma la lista de 15 predictores originales.
- Muestra la duración media de la llamada según si el cliente contrató o no.

Por qué

- Es clasificación binaria supervisada. Se excluye **duracion** porque solo se conoce al terminar la llamada: usarla sería fuga de información. La tabla de medias lo demuestra (554 s en quienes contratan vs 228 s en quienes no).
- id_cliente y pliegue no describen al cliente: son un identificador y una etiqueta de diseño.

Resultado clave: 15 predictores; duracion excluida por fuga.

1(b) Exploración de calidad de datos

Qué hace el código

- Separa variables numéricas y categóricas y cuenta valores faltantes (NA).
- Cuenta las categorías "unknown" por variable, las filas duplicadas y los id repetidos.
- Calcula el balance de clases total y por pliegue.
- Cuenta saldos negativos y verifica que `dias_previos = -1` coincide exactamente con no tener contactos previos.
- Calcula la exactitud de predecir siempre "no contrata".

Por qué

- Antes de modelar hay que saber con qué se trabaja: no hay NA pero sí "unknown" (en resultado_previo, el 82 % significa "sin campaña previa").
- El saldo negativo es un sobregiro (dato válido) y el -1 es un código, no un número de días.
- Con 88,5 % de "no", la exactitud es engañosa: predecir siempre "no" acierta 88,5 % pero no encuentra a ningún cliente que contrate.

Resultado clave: 11,5 % contrata; exactitud de la clase mayoritaria = 88,5 % con sensibilidad 0.

1(c) Variables nuevas, one-hot y escalamiento

Qué hace el código

- `construir_variables()`: crea `contactado_antes` (1 si hubo campaña previa) y `log_contactos_previos` = log(1 + contactos previos), y reemplaza -1 por 0 en dias_previos.
- `nuevo_preparador()`: un ColumnTransformer que estandariza las 6 numéricas, deja el indicador binario tal cual y aplica one-hot a las 9 categóricas (conservando "unknown", con `drop='first'` y `handle_unknown='ignore'`).
- Lo ajusta una vez en todo el entrenamiento solo para mostrar las 42 columnas resultantes.

Por qué

- El indicador separa "nunca contactado" de "contactado hace X días"; el logaritmo reduce la asimetría (5,85 a 2,51) y el peso de valores extremos.
- drop='first' deja una categoría de referencia: sin eso la regresión logística sin penalización tendría columnas perfectamente redundantes.
- Estandarizar es imprescindible para SVM (distancias) y útil para la logística; árboles y Naive Bayes no lo necesitan, pero no les afecta.

- Las transformaciones son funciones: se vuelven a ajustar dentro de cada pliegue para evitar fuga.

Resultado clave: p = 42 columnas preparadas.

Pregunta 2 · Selección de variables y modelos

2(a) Función de validación y regresión logística con y sin selección

Qué hace el código

- `metricas()`: calcula log loss, Brier y AUC (recorta probabilidades a [1e-15, 1-1e-15] para evitar log(0)).
- `validar()`: el corazón del notebook. Para cada uno de los 5 pliegues entregados ajusta el preparador *solo* con las filas de ajuste, transforma ajuste y validación, opcionalmente selecciona las 10 columnas con mayor F de ANOVA (también aprendido en ajuste), entrena el modelo y guarda la predicción del pliegue que quedó fuera. Para SVM guarda el puntaje de decisión en vez de la probabilidad.
- `logistica_sin_penalizacion()`: usa C=inf en scikit-learn 1.8 o superior y penalty=None en versiones anteriores (Colab).
- Corre la logística con 42 columnas y con 10 columnas, muestra qué columnas se eligieron en cada pliegue y compara métricas.

Por qué

- Centralizar la validación en una función garantiza que todos los modelos se comparan exactamente igual y sin fuga.
- Mostrar la selección por pliegue permite ver si es estable (lo es) y si es redundante (lo es: varias columnas dicen "hubo campaña previa").

Resultado clave: Logística 42 columnas: log loss 0,315, AUC 0,725; con 10 columnas: 0,320 y 0,701. Seleccionar empeora.

2(b) Comparación de todos los modelos

Qué hace el código

- Define los modelos de la tabla del enunciado: árboles (profundidad 3 y 6, mín. 20 por hoja), Random Forest (200 árboles, sqrt(p) columnas, mín. 10 por hoja), Gradient Boosting (200 árboles, profundidad 3, tasa 0,05), Naive Bayes gaussiano y SVM radial (C=1, gamma=1/p).
- Los pasa todos por validar() y crea el modelo de referencia: predice la proporción de "sí" en las filas de ajuste de cada pliegue.
- Arma la tabla de log loss, Brier y AUC (SVM solo AUC); para los árboles agrega el log loss medio de entrenamiento.
- Guarda en `MODELO_ELEGIDO` el de menor log loss OOF.

Por qué

- El log loss premia probabilidades bien calibradas, que es lo que se necesita para decidir con costos; por eso es el criterio de selección.
- La referencia muestra cuánto aporta realmente cada modelo; el log loss de entrenamiento de los árboles permite medir sobreajuste.

Resultado clave: Gana Gradient Boosting (log loss 0,311; AUC 0,744). Referencia: 0,357. Naive Bayes: 2,256 (peor que la referencia).

2(c) Evidencia para interpretar

Qué hace el código

- Tabla de brecha entrenamiento vs OOF de los árboles.
- Revisa cuántas probabilidades de Naive Bayes son extremas (< 0,01 o > 0,99) y compara la probabilidad media de cada modelo con la tasa real.

- Gráfico de calibración por deciles y curvas ROC de todos los modelos.
- Grilla de SVM con C en {0,1; 1; 10} y gamma en {0,1/p; 1/p; 10/p}, y una SVM sin estandarizar.

Por qué

- Las preguntas teóricas se responden mejor con datos propios: la brecha mayor del árbol 6 muestra sobreajuste; las probabilidades extremas de Naive Bayes muestran el efecto de suponer independencia; la SVM sin escalar (AUC 0,60 vs 0,69) muestra por qué hay que estandarizar.
- La calibración vs ROC muestra que ordenar bien (AUC) no implica probabilidades buenas (log loss).

Pregunta 3 · Evaluación y decisión

3(a) Elegir el umbral por costo

Qué hace el código

- Con las probabilidades OOF del modelo elegido recorre umbrales 0,05 a 0,95; para cada uno calcula TP, FP, FN, TN, sensibilidad, especificidad y C = FP + 5·FN.
- Guarda en `UMBRAL` el de menor costo y grafica la curva de costo.

Por qué

- Perder un cliente que contrataría (FN) cuesta 5 veces más que una llamada inútil (FP); por eso conviene un umbral bajo. En teoría, con probabilidades bien calibradas, el óptimo es 1/(1+5) ≈ 0,167.
- Se usa OOF y no la prueba para que la elección no esté sesgada.

Resultado clave: Umbral elegido 0,15 (costo OOF 1492, contra 1816 con 0,50).

3(b) Evaluación final en el conjunto de prueba

Qué hace el código

- Imprime el modelo y umbral ya decididos, recién entonces lee la prueba y le aplica las mismas variables nuevas.
- Reajusta el preparador y el modelo con todo el entrenamiento y predice la prueba; la referencia usa la proporción de todo el entrenamiento.
- Calcula métricas, matrices de confusión con t = 0,15 y t = 0,50, sensibilidad, especificidad, C/n, la curva ROC y la tasa de contratación por decil.

Por qué

- Es la estimación honesta del desempeño futuro. Comparar con 0,50, con la referencia y con "llamar a todos" muestra el valor del umbral y del modelo.
- La tabla por deciles traduce el modelo a algo accionable: a quién llamar primero.

Resultado clave: Prueba: log loss 0,309, AUC 0,750. Con t = 0,15: C/n = 0,395 (0,477 con 0,50; 0,575 la referencia); detecta 53 % de los que contratan llamando al 18 %.

Pregunta 4 · PCA y agrupamiento

4(a) Componentes principales

Qué hace el código

- Estandariza edad, saldo, contactos_campana y contactos_previos con media y desviación muestral (ddof=1).
- Ajusta PCA, informa la varianza explicada y las cargas, la matriz de correlación y un gráfico PC1–PC2 coloreado por contrata, con flechas de cargas.

Por qué

- Sin estandarizar, saldo (en euros) dominaría las componentes. La matriz de correlación explica el resultado: las variables casi no se correlacionan, así que PCA no logra comprimir mucho.

Resultado clave: PC1 27,7 % + PC2 26,7 % = 54,4 %.

4(b) K-means

Qué hace el código

- Ajusta K-means con k = 2, 3 y 4 (30 inicializaciones, semilla 2026), calcula WSS y silueta media y elige el k de mayor silueta.
- Describe cada grupo (tamaño, medias, mediana del saldo, % con contacto previo, tasa de contratación), muestra los centroides y los grafica sobre el plano PCA.

Por qué

- El WSS siempre baja al aumentar k, por eso se usa la silueta para elegir.
- La tasa de contratación se calcula *después*, como descripción: K-means no usa la respuesta, así que los grupos no son predicciones.

Resultado clave: k = 4 (silueta 0,363). Grupo con campañas previas: 26,6 % contrata; grupo muy insistido: 4,8 %.

Pregunta 5 · Detección de anomalías

5(a) Distancia de Mahalanobis

Qué hace el código

- Calcula la matriz de covarianza de las 4 variables estandarizadas, su inversa y la distancia al cuadrado d^2 de cada registro.
- Usa como corte el cuantil 0,99 de una chi-cuadrado con 4 grados de libertad, cuenta los marcados y muestra los dos de mayor distancia (valores originales y estandarizados).

Por qué

- Mahalanobis mide qué tan lejos está un registro del centro considerando la escala y correlación de las variables. Si los datos fueran normales, d^2 seguiría una chi-cuadrado con 4 g.l., y el corte marcaría al 1 %.

Resultado clave: Corte 13,28; marcados 189 (5,23 %). Top 2: id 1477 (saldo 71.188 €) e id 1542 (50 contactos).

5(b) Interpretar los atípicos

Qué hace el código

- Muestra percentiles de las 4 variables y la posición de cada uno de los dos registros, y su fila completa.
- Calcula asimetría y curtosis, compara el % esperado (1 %) con el observado y la tasa de contratación de marcados vs no marcados.

Por qué

- Permite argumentar que cada registro es extremo en una sola variable, que eso no prueba un error (pueden ser casos reales) y que el corte no garantiza 1 % porque las variables no son normales (colas muy pesadas). Sirve también para decidir qué revisar antes de eliminar un registro.

Antes de entregar

- Completar número de grupo e integrantes en la primera celda.
- Ejecutar todo en Colab (Entorno de ejecución → Ejecutar todas) y revisar que las cifras de las respuestas coincidan; con otra versión de scikit-learn pueden variar en el tercer decimal.

- Descargar el .ipynb ejecutado (Archivo → Descargar → .ipynb) y comprimirlo con los dos CSV en Grupo_XX_Taller3.zip.